

Minimum Maximal Acyclic Matchings in Connected Cubic Graphs: Exhaustive Enumeration and a Sharp Bound

Hao Tang

Received: date / Accepted: date

Abstract An acyclic matching is a matching whose endpoints induce a forest. Given a graph G , the problem LOW-ACY-MATCHING asks for an inclusion-wise maximal acyclic matching of minimum cardinality; we write $\mu'_{\text{ac}}(G)$ for this optimum. The decision version is NP-complete on general graphs. We determine the exact extremal value of μ'_{ac} over connected simple cubic graphs.

Fürst and Rautenbach proved that the maximum acyclic matching number satisfies $\nu_{\text{ac}}(G) \geq n/4$ on connected cubic graphs other than $K_{3,3}$. The corresponding $\lfloor n/4 \rfloor$ benchmark holds for μ'_{ac} through order 14, but fails at order 16: four of 4060 non-isomorphic graphs have $\mu'_{\text{ac}}(G) = 3$. For a maximal acyclic matching M , analysis of the unmatched induced subgraph and its incidence with the forest induced by $V(M)$ yields $n \leq 5|M|$ when $|M|$ is even and $n \leq 5|M| + 1$ when $|M|$ is odd. Explicit connected cubic constructions attain the bound at every even order. Thus, if $f_3(n)$ denotes the minimum of $\mu'_{\text{ac}}(G)$ over connected simple cubic graphs of order n , then $f_3(n) = \lceil (n-1)/5 \rceil$ for every even $n \geq 4$.

Two independent exact solvers, dual regular-graph generators, and an endpoint-set audit certify the census through order 16. As a direct counting consequence, deciding $\mu'_{\text{ac}}(G) \leq K$ on this class reduces immediately to instances with at most $5K + 1$ vertices, and is therefore fixed-parameter tractable in the solution-size parameter K .

Keywords Acyclic matching · Cubic graph · Extremal graph theory · Exhaustive enumeration · Parameterized algorithm

H. Tang
College of Software, Jilin University, Changchun 130012, China
E-mail: tanghao5524@mails.jlu.edu.cn

1 Introduction

Matching is a fundamental problem in combinatorial optimization and graph algorithms. Classical maximum matching is polynomial-time solvable Edmonds (1965); Lovász and Plummer (2009), whereas the complexity often increases sharply when feasibility is defined through an induced-subgraph property or when the objective is changed from a maximum feasible solution to a minimum maximal one. Let G be a finite simple graph and let $M \subseteq E(G)$ be a matching. The matching M is *acyclic* if its endpoints induce a forest in G . Acyclic matchings were developed systematically within the framework of generalized subgraph-restricted matchings Goddard et al. (2005), and their maximum and minimum-maximal variants have since led to distinct algorithmic lines of research Chaudhary et al. (2024, 2025); Fürst and Rautenbach (2019); Panda and Chaudhary (2023); Chaudhary and Zehavi (2025).

This paper studies LOW-ACY-MATCHING: among all inclusion-wise maximal acyclic matchings, find one of minimum cardinality. Its optimum is denoted by $\mu'_{\text{ac}}(G)$. This problem differs from maximum acyclic matching and from ordinary minimum maximal matching, which is closely related to edge domination Yannakakis and Gavril (1980). Induced and uniquely restricted matchings provide further nearby reference points Cameron (1989); Golumbic et al. (2001), and the minimum-maximal variant of uniquely restricted matching exhibits the same min-max pattern studied here Chaudhary and Panda (2021). The decision version of LOW-ACY-MATCHING is NP-complete on general graphs. For restricted classes, it remains NP-complete on bipartite graphs of maximum degree six and is APX-hard on 4-regular graphs, whereas proper interval graphs admit a linear-time algorithm Chaudhary et al. (2024, 2025). These results do not settle the per-instance complexity on connected cubic graphs. The present paper addresses first a different, extremal question: the minimum possible value of $\mu'_{\text{ac}}(G)$ among connected cubic graphs of a fixed order. Chaudhary et al. (2024) observed that even a general lower bound was missing for fixed r -regular graphs with $r \geq 3$. Cubic graphs are therefore the first unresolved regular class in which to seek a sharp universal bound.

The corresponding question for the *maximum* acyclic matching number ν_{ac} is settled. Fürst and Rautenbach (2018) proved that every subcubic graph without isolated vertices satisfies

$$\nu_{\text{ac}}(G) \geq \frac{1}{4}(n - \kappa_G(K_{2,3}) - \kappa_G(K_4^+) - 2\kappa_G(K_{3,3})),$$

where $\kappa_G(H)$ counts the components of G isomorphic to H and K_4^+ arises from K_4 by subdividing one edge once. Neither $K_{2,3}$ nor K_4^+ is cubic, so the bound specialises to $\nu_{\text{ac}}(G) \geq n/4$ for every connected cubic graph other than $K_{3,3}$, and it is tight. Since $\mu'_{\text{ac}}(G) \leq \nu_{\text{ac}}(G)$, this inequality points in the wrong direction for the minimum-maximal parameter and settles nothing about μ'_{ac} . It does, however, make $\lfloor n/4 \rfloor$ the natural first guess for $f_3(n)$, and our census shows that the guess is wrong: the quarter threshold that governs the maximum variant on cubic graphs does not govern the minimum-maximal one, whose true growth rate is $n/5$.

We follow an exact-computation-to-structure route: discover counterexamples computationally, formulate a structural conjecture, and then determine the extremal function by an independent proof. We first implement a definition-level solver and an optimized bitset solver, and then enumerate small non-isomorphic connected cubic graphs. The census through order 14 happens to satisfy the benchmark $\mu'_{\text{ac}}(G) \geq \lfloor n/4 \rfloor$ inherited from ν_{ac} , but the order-16 census refutes it and identifies four smallest counterexamples. Their structure shows that the decisive feature is not the local form of the matching edges, but how every edge in the subgraph induced by unmatched vertices is blocked by connected components of the forest induced by the matching endpoints. This observation yields a local characterization, a component classification, and an exact cut-edge count.

1.1 Main results and computational validation

The contributions are as follows.

- For every maximal acyclic matching M with $k = |M|$, we prove $n \leq 5k$ when k is even and $n \leq 5k + 1$ when k is odd. The split identifies precisely when the one-vertex slack can occur.
- Equality constructions are given for every even $n \geq 4$, determining the exact extremal function

$$\begin{aligned} f_3(n) &= \min\{\mu'_{\text{ac}}(G) : G \text{ connected simple cubic of order } n\} \\ &= \left\lceil \frac{n-1}{5} \right\rceil. \end{aligned}$$

- All non-isomorphic connected simple cubic graphs of orders $n = 4, 6, \dots, 16$ are evaluated exactly. The complete histogram exposes four of the 4060 order-16 graphs with $\mu'_{\text{ac}}(G) = 3 < 4$ and leads to the sharp theorem. Exactly three of these four are realised by the equality construction of Section 6; the fourth is not, so extremal graphs are not confined to a single structural family.
- For reproducibility, a definition-level edge-subset solver and a bitset backtracking solver are specified and proved correct. Their results are cross-checked through order 12, while a structurally independent endpoint-set solver certifies every optimum at orders 14 and 16. GENREG and nauty/geng produce identical canonical input censuses.
- As a post-theorem consequence, we derive an exact solver that contracts selected matching edges, maintains an edge-interaction forest with rollback union-find, and uses a theorem-derived defect budget. The same counting bound implies that deciding $\mu'_{\text{ac}}(G) \leq K$ is fixed-parameter tractable in K on connected cubic graphs: every yes-instance has at most $5K + 1$ vertices, so larger instances are immediate no-instances.

The last item is deliberately downstream of the theorem. The new solver uses the proved lower bound and therefore is not counted as independent evidence for that bound.

1.2 Organization

Section 2 gives the definitions, two theorem-independent exact algorithms, and their correctness analyses. Section 3 reports the non-isomorphic census and counterexample certificates. Section 4 establishes the local blocking structure, and Section 5 proves the sharp lower bound. Section 6 gives equality constructions for all even orders. Section 7 develops the post-theorem interaction-forest solver and records the FPT consequence of the bound. Section 8 describes independent verification, reproducibility, and threats to validity. Sections 9 and 10 discuss open questions and conclude the paper. Detailed Solver C timings, equality-family scaling, and ablations are collected in Section A.

2 Problem definition and exact algorithms

2.1 Definitions and decision semantics

All graphs considered in this paper are finite, undirected, and simple. For $X \subseteq V(G)$, let $G[X]$ denote the vertex-induced subgraph of G on X . For a matching M , let $V(M)$ be the set of all endpoints of edges in M .

Definition 2.1. A matching M is *acyclic* if $G[V(M)]$ is a forest. It is *maximal acyclic* if it is not strictly contained in any larger acyclic matching. Define

$$\mu'_{\text{ac}}(G) = \min\{|M| : M \text{ is a maximal acyclic matching of } G\}.$$

For every even integer $n \geq 4$, define

$$f_3(n) = \min\{\mu'_{\text{ac}}(G) : G \text{ is a connected simple cubic graph of order } n\}.$$

A common implementation error is to interpret an extension as merely adding a new edge uv to $G[V(M)]$. After uv is added to the matching, however, one must test the full vertex-induced subgraph $G[V(M) \cup \{u, v\}]$, including every edge from u and v to the previous matching endpoints. All exact solvers in this work implement this semantics.

2.2 Solver A: definition-level enumeration

Solver A exhausts every candidate of smaller cardinality before advancing to the next cardinality, so the first returned solution is optimal. Deliberately avoiding sophisticated pruning, it serves as a definition-level baseline and as an independent cross-check. If $m = |E(G)|$, its worst-case running time is $\mathcal{O}(2^m \text{poly}(n + m))$, so it is suitable only for small graphs.

Proposition 2.2. *Solver A returns a minimum-cardinality maximal acyclic matching.*

Algorithm 1 Solver A: definition-level exact enumeration

Input: A finite simple graph $G = (V, E)$

```

1: for  $k = 0, 1, \dots, |E|$  do
2:   for all  $k$ -edge subsets  $M \subseteq E$  do
3:     if  $M$  is not a matching then
4:       continue
5:     if  $G[V(M)]$  is not a forest then
6:       continue
7:      $U \leftarrow V \setminus V(M)$ ;  $\text{maximal} \leftarrow \text{TRUE}$ 
8:     for all  $uv \in E(G[U])$  do
9:       if  $G[V(M) \cup \{u, v\}]$  is a forest then
10:         $\text{maximal} \leftarrow \text{FALSE}$ ; break
11:     if  $\text{maximal}$  then
12:       return  $(k, M)$ 

```

Proof. The algorithm returns only if M is a matching, $G[V(M)]$ is a forest, and no edge $uv \in E(G[V \setminus V(M)])$ extends it while preserving the induced forest condition. Thus the returned solution is feasible and maximal. The outer loop increases k only after all smaller edge subsets have been checked, so no smaller solution exists. $\square$

2.3 Solver B: bitset increasing-cardinality search

Index the edges as $E = (e_0, \dots, e_{m-1})$ and let $p[i]$ be the bit mask of the two endpoints of e_i . Solver B searches in increasing target size and caches the exact predicate $\text{Forest}(X)$, which tests whether $G[X]$ is a forest. For a forest $G[X]$, the cache stores both acyclicity and its component count. Cyclicity is monotone under adding vertices to an induced subgraph, so a failed forest test safely prunes the entire branch. Two capacity tests are also safe: a completion needs at least *need* eligible suffix edges and 2 *need* distinct endpoints.

Proposition 2.3. *Solver B returns a minimum-cardinality maximal acyclic matching.*

Proof. At every recursive call, L is an acyclic matching, $X = V(L)$, and the edge indices in L are strictly smaller than *start*. An endpoint conflict cannot occur in a matching. If too few eligible suffix edges or too few distinct suffix endpoints remain, no completion exists. If $G[X']$ contains a cycle, every induced supergraph on a superset of X' contains that cycle. Thus all pruning rules are safe.

Conversely, list the edge indices of any acyclic k -edge matching in increasing order. The recursion can follow these indices. None conflicts with earlier endpoints, none fails the exact forest test, and the unused edges of the matching witness both capacity requirements. Hence every feasible k -edge matching is reached exactly once.

At depth k , $\text{MAXIMAL}(X)$ tests every edge with both endpoints outside X . It returns true exactly when every one-edge induced extension contains a cycle.

Algorithm 2 Solver B: bitset- and cache-driven exact search

Input: $G = (V, E)$ with endpoint masks $p[0], \dots, p[m-1]$

```

1: function MAXIMAL( $X$ )
2:   for  $i = 0, \dots, m-1$  do
3:     if  $(p[i] \& X) = 0$  and Forest( $X \mid p[i]$ ) then
4:       return FALSE
5:   return TRUE
6: function SEARCH( $start, q, X, L, k$ )
7:   if  $q = k$  then
8:     return  $L$  if MAXIMAL( $X$ ); otherwise failure
9:    $need \leftarrow k - q$ 
10:   $R \leftarrow \{i \geq start : (p[i] \& X) = 0\}$ 
11:  if  $|R| < need$  then
12:    return failure
13:   $Y \leftarrow \bigvee_{i \in R} p[i]$ 
14:  if popcount( $Y$ )  $< 2 \cdot need$  then
15:    return failure
16:  for all  $i \in R$  in increasing order do
17:     $X' \leftarrow X \mid p[i]$ 
18:    if  $\neg$ Forest( $X'$ ) then
19:      continue
20:     $A \leftarrow$  SEARCH( $i+1, q+1, X', L \cup \{e_i\}, k$ )
21:    if  $A \neq$  failure then
22:      return  $A$ 
23:  return failure
24: for  $k = 0, 1, \dots, \lfloor n/2 \rfloor$  do
25:    $A \leftarrow$  SEARCH( $0, 0, 0, \emptyset, k$ )
26:   if  $A \neq$  failure then
27:     return  $(k, A)$ 
  
```

This is equivalent to inclusion-wise maximality because acyclicity is hereditary. The outer loop exhausts all smaller cardinalities first. $\square$

The worst-case search remains exponential. Solver B contributes an exact, auditable implementation with strong practical pruning, not a new worst-case exponential base.

3 Exhaustive enumeration of non-isomorphic cubic graphs

3.1 Graph generation and experimental protocol

A cubic graph has even order. The experiments process $n = 4, 6, 8, 10, 12, 14, 16$. To remove dependence on one generation algorithm, every order was generated independently by GENREG, which implements Meringer’s orderly regular-graph generation method Meringer (1999), and by nauty 2.9.3 **geng** with the restrictions **-c -d3 -D3** and exactly $3n/2$ edges McKay and Piperno (2014). GENREG short-code records were decoded by a separate parser. Both collections were converted to **graph6** and rechecked for simplicity, connectivity, and degree three.

Table 1: Exact census of connected simple cubic graphs through order 16. The benchmark column is $\lfloor n/4 \rfloor$.

n	Classes	Min.	Max.	Benchmark	At min. / violations
4	1	1	1	1	1 / 0
6	2	1	1	1	2 / 0
8	5	2	2	2	5 / 0
10	19	2	3	2	14 / 0
12	85	3	3	3	85 / 0
14	509	3	4	3	418 / 0
16	4060	3	4	4	4 / 4

Table 2: Complete histogram of μ'_{ac} over the cubic census.

n	μ'_{ac}	Classes	n	μ'_{ac}	Classes
4	1	1	12	3	85
6	1	2	14	3	418
8	2	5	14	4	91
10	2	14	16	3	4
10	3	5	16	4	4056

Each collection was canonically labelled with nauty `labelg`, sorted, and compared as a multiset. At orders 4, 6, 8, 10, 12, 14, 16, the two generators produced 1, 2, 5, 19, 85, 509, 4060 distinct graphs, respectively; the canonical multisets were identical at every order, with no missing, additional, or duplicated class. These well-known census counts are completeness checks rather than new enumeration results. Canonicalization is used only for comparison and is not part of either generation procedure.

The solver protocol has three levels. First, definition-level unit tests cover K_4 , $K_{3,3}$, paths, cycles, and small graph-atlas instances. Second, Solvers A and B are compared graph by graph on every cubic graph of order at most 12. Third, an independently written endpoint-set solver computes the optimum of every one of the 509 order-14 and 4060 order-16 graphs. It enumerates vertex sets S in increasing even cardinality, uses a disjoint-set union test for $G[S]$, independently searches for a perfect matching of $G[S]$, and directly tests every two-vertex induced extension. The experiment terminates immediately on any discrepancy.

3.2 Complete census

Table 1 summarises the outcome and Table 2 gives the full histogram. For $n \leq 14$, every graph satisfies the $\lfloor n/4 \rfloor$ benchmark. The first violations occur at order 16: 4056 of the 4060 graphs have parameter value four, while four have value three. Hence $f_3(16) = 3 < 4 = \lfloor 16/4 \rfloor$.

Table 3: All four counterexamples of order 16. “Diam.”, “Br.”, “Tri.”, and C_4 denote diameter, number of bridges, number of triangles, and number of 4-cycles, respectively.

ID	Optimal matching	Diam.	Br.	Tri.	C_4
4055	$\{(10, 12), (0, 4), (5, 6)\}$	9	2	6	7
4056	$\{(0, 4), (5, 6), (10, 11)\}$	9	2	4	11
4058	$\{(0, 4), (5, 6), (11, 15)\}$	8	2	5	8
4059	$\{(0, 4), (5, 6), (11, 12)\}$	6	3	6	9

Table 4: **graph6** encodings of the four counterexamples.

ID	graph6 encoding
4055	0~o?GGB?w????B@@`@??X
4056	0~o?GGB?w??@?A@@`@??L
4058	0~o?GGB?w??@?B@?`@??J
4059	0~o?GGB?wW?@?@?@?W?F

3.3 The four order-16 counterexamples

The **graph6** encoding of the first counterexample is 0~o?GGB?w????B@@`@??X. Take

$$M = \{(0, 4), (5, 6), (10, 12)\}.$$

The subgraph induced by $V(M)$ is exactly $3K_2$ and is therefore a forest. The unmatched induced subgraph contains nine edges; for every such edge uv , the graph $G[V(M) \cup \{u, v\}]$ contains a triangle or a 4-cycle. Thus M cannot be extended. Solver A exhausts cardinalities zero, one, and two, and Solver B returns the same optimum. The independent endpoint-set audit also returns three for exactly the four canonical classes listed in Table 3, whose **graph6** encodings are recorded in Table 4. Figure 2 illustrates the three blocking mechanisms established below.

The four counterexamples are pairwise non-isomorphic, non-bipartite, contain triangles, and have vertex connectivity one. Their common structure motivates the analysis of the components of the unmatched induced subgraph. In all four, the optimal matching satisfies $c = 3$ and $t = 1$ in the notation of Section 4, and the induced forest $G[V(M)]$ is $3K_2$; the unmatched induced subgraph is $2C_3 \cup P_4$ for IDs 4055, 4056, and 4058, and $3C_3 \cup K_1$ for ID 4059.

This split is exactly the boundary of the equality family. The existence proof in Section 6 uses one canonical assignment of degree slots, recorded there. Independently enumerating all bijections of the $t = 1$ base construction yields 3456 labelled graphs falling into precisely three isomorphism classes, namely IDs 4055, 4056, and 4058. The graph in Figure 1 is one of them. ID 4059 is not obtained by any slot assignment: it is the graph formed by three leaf pieces $K_2 + C_3$ whose three K_2 components each send one degree slot to a single common unmatched vertex. Its unmatched induced subgraph therefore has

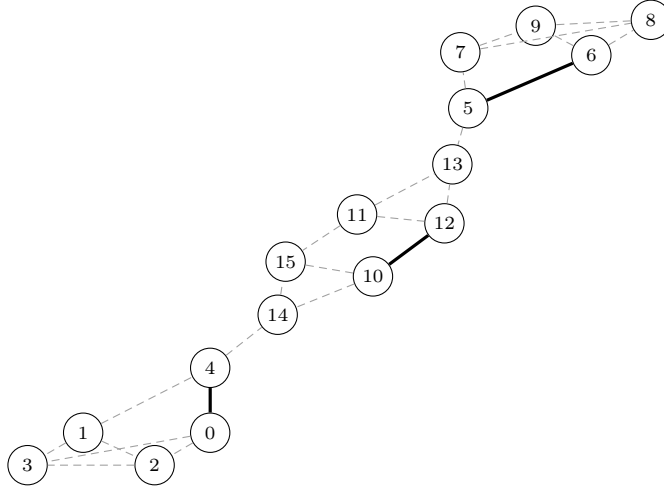


Fig. 1: A 16-vertex cubic graph with $\mu'_{ac}(G) = 3 < 4$. Thick edges form the matching; dashed edges are the remaining graph edges.

an isolated vertex as its unique tree component instead of the root P_4 , and it has three bridges rather than two. Consequently the construction of Section 6 is sufficient for sharpness but is not a classification of extremal graphs; see Remark 6.2.

4 Local blocking structure of maximality

Fix a connected simple cubic graph G and an arbitrary maximal acyclic matching M . Let

$$k = |M|, \quad S = V(M), \quad U = V(G) \setminus S, \quad F = G[S], \quad H = G[U].$$

By definition, F is a forest and $|S| = 2k$. Note that $k \geq 1$: the empty matching is acyclic but never maximal, because a single edge always induces a forest. Hence $c \geq 1$ throughout, and $U \neq \emptyset$, since otherwise $F = G$ would be a cubic forest.

4.1 Maximality as non-extendability by one edge

Lemma 4.1. *An acyclic matching M is maximal if and only if there is no edge $e \in E(G) \setminus M$ such that $M \cup \{e\}$ is an acyclic matching.*

Proof. If an acyclic matching M' strictly contains M , choose $e \in M' \setminus M$. Acyclic matchings are closed under taking subsets, so $M \cup \{e\}$ remains acyclic. Hence the absence of a one-edge extension is equivalent to the absence of every strict extension. $\square$

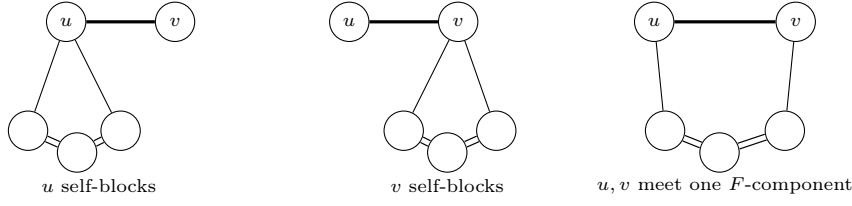


Fig. 2: The three local mechanisms that block an unmatched edge. Double strokes mark paths that lie in a component of the forest F ; these remain legible in greyscale.

An edge can be added to M exactly when both endpoints lie in U ; the candidate extension edges are precisely $E(H)$. Consequently, for every $uv \in E(H)$, the induced subgraph $G[S \cup \{u, v\}]$ contains a cycle.

4.2 An exact local blocking criterion

For $x \in U$ and a component C of F , define

$$d_C(x) = |N_G(x) \cap V(C)|.$$

Lemma 4.2. *Let $uv \in E(H)$. Then $G[S \cup \{u, v\}]$ contains a cycle if and only if at least one of the following holds:*

- (i) $d_C(u) \geq 2$ for some component C of F ;
- (ii) $d_C(v) \geq 2$ for some component C of F ;
- (iii) u and v are both adjacent to the same component C of F .

Proof. If (i) holds, two distinct neighbors of u in the tree C are joined by a unique path in C ; that path together with u forms a cycle. The proof for (ii) is symmetric. If (iii) holds, choose $a \in N(u) \cap V(C)$ and $b \in N(v) \cap V(C)$. If $a = b$, then uav is a triangle. Otherwise the unique a - b path in C , together with ua, uv, vb , is a cycle.

Conversely, let Q be a cycle in $G[S \cup \{u, v\}]$. Since F is a forest, Q contains u or v . If it contains exactly one, that vertex has two neighbors on Q in one component of F , giving (i) or (ii). If Q contains both u and v , either deleting uv from Q , or taking either u - v arc when $uv \notin E(Q)$, gives a path whose internal vertices lie in one F -component. Thus (iii) holds. $\square$

4.3 Components of the unmatched induced subgraph

Lemma 4.3. *Every connected component of $H = G[U]$ is a path, a cycle, or $K_{1,3}$.*

Proof. Suppose $x \in U$ has degree three in H . Since G is cubic, x has no neighbor in S . For any $y \in N_H(x)$, apply Lemma 4.2 to xy . The vertex x cannot block the edge by itself, and x, y cannot be jointly adjacent to an F -component. Hence y must block the edge by itself. It has exactly two neighbors in one F -component and, by cubicity, x is its only neighbor in H . Thus the component containing x is $K_{1,3}$.

If an H -component contains no vertex of degree three, its maximum degree is at most two. Every finite connected graph of maximum degree at most two is a path or a cycle. $\square$

Lemma 4.4. *Every tree component of H is adjacent to at most three distinct components of F , whereas every cycle component of H is adjacent to exactly one component of F .*

Proof. An isolated vertex of H has three neighbors in S and hence meets at most three F -components. If the component is $K_{1,3}$, its center has no neighbor in S . Applying Lemma 4.2 to each center-leaf edge shows that the two S -neighbors of each leaf lie in one F -component; the three leaves therefore meet at most three such components.

Consider a path $x_1 \dots x_r$. The case $r = 2$ is immediate: if one endpoint self-blocks, it contributes one F -component and the other at most two; if neither self-blocks, their two sets of incident components intersect, so their union has size at most three. For $r \geq 3$, every internal vertex has exactly one neighbor in S and cannot self-block. Applying Lemma 4.2 along the edges between consecutive internal vertices shows that all internal vertices meet one common F -component C . Each endpoint contributes at most one component outside C , so the whole path meets at most three components. The same argument also covers P_3 .

Finally, every vertex in a cycle component of H has exactly one neighbor in S and cannot self-block. Each cycle edge must therefore be blocked through a common F -component. Propagating around the cycle shows that all its vertices meet one and the same component of F . $\square$

4.4 The component-incidence graph

Let c be the number of connected components of F , and let t be the number of tree components of H , including paths, isolated vertices, and $K_{1,3}$ components. Construct a simple bipartite graph B whose left side consists of the F -components and whose right side consists of the H -components. Two component vertices are adjacent when at least one edge of G joins the corresponding components.

Lemma 4.5. *The inequalities $c \leq 2t + 1$ and $c \leq k$ hold.*

Proof. Connectivity of G implies that B is connected. By Lemma 4.4, every cycle component of H is a leaf of B . Delete all such leaves to obtain B_0 . Deleting leaves cannot disconnect the remaining vertices. Hence B_0 is connected.

If $t = 0$, connectedness forces $c = 1$, so $c \leq 2t + 1$. If $t \geq 1$, then B_0 has $c + t$ vertices and at least $c + t - 1$ edges. Each of its t right-side vertices has degree at most three by Lemma 4.4, so $|E(B_0)| \leq 3t$. Therefore $c + t - 1 \leq 3t$, or $c \leq 2t + 1$.

Every component of F contains at least one edge of M , and distinct F -components contain distinct matching edges. Thus $c \leq k$. $\square$

5 Sharp lower bound

5.1 Cut-edge counting

The forest F has $2k$ vertices and c components, so $|E(F)| = 2k - c$. Counting the cut $E(S, U)$ from the S side gives

$$|E(S, U)| = 3|S| - 2|E(F)| = 6k - 2(2k - c) = 2k + 2c. \quad (1)$$

By Lemma 4.3, every component of H is a tree or a cycle. A tree component has one fewer edge than vertices, while a cycle component has as many edges as vertices. Since H has t tree components, $|E(H)| = |U| - t$. Counting the same cut from the U side gives

$$|E(S, U)| = 3|U| - 2|E(H)| = 3|U| - 2(|U| - t) = |U| + 2t. \quad (2)$$

Equating (1) and (2) yields

$$|U| = 2k + 2c - 2t, \quad n = |S| + |U| = 4k + 2(c - t). \quad (3)$$

5.2 Integer optimization

Subtracting t from the bounds in Lemma 4.5 gives

$$c - t \leq \min\{k - t, t + 1\}.$$

For integral $t \geq 0$, the maximum of the right-hand side is $\lceil k/2 \rceil$. Substituting this into (3) gives

$$n \leq 4k + 2 \left\lceil \frac{k}{2} \right\rceil = \begin{cases} 5k, & k \text{ even}, \\ 5k + 1, & k \text{ odd}. \end{cases}$$

Theorem 5.1. *Let G be a connected simple cubic graph of order n , and let M be any maximal acyclic matching of G with $k = |M|$. Then $n \leq 5k$ when k is even and $n \leq 5k + 1$ when k is odd. In particular,*

$$\mu'_{\text{ac}}(G) \geq \left\lceil \frac{n-1}{5} \right\rceil.$$

Proof. The parity-sensitive inequalities follow from (3) and the integer optimization above. Both imply $n \leq 5k + 1$, and hence $k \geq (n - 1)/5$. Since k is integral, $k \geq \lceil (n - 1)/5 \rceil$. The argument applies to every maximal acyclic matching, and therefore to one of minimum cardinality. $\square$

Because every cubic graph has even order, the two parity cases are numerically equivalent to the unified inequality $n \leq 5k + 1$: when k is even, the right-hand side is odd and integrality sharpens it to $n \leq 5k$. The split remains structurally informative, since the extra vertex is possible only for odd k , as the example $n = 16, k = 3$ demonstrates.

6 Equality constructions for every even order

Throughout this section, P_r and C_r denote the path and cycle on r vertices. For $P_{2s} = v_1 \dots v_{2s}$, its alternating matching is

$$\{v_1v_2, v_3v_4, \dots, v_{2s-1}v_{2s}\}.$$

For each even $n \geq 4$, we construct a connected simple cubic graph of order n together with a maximal acyclic matching of size $\lceil (n - 1)/5 \rceil$. By Theorem 5.1, every displayed matching is automatically minimum.

6.1 Base construction for odd matching size

Let $k_0 = 2t + 1$ be odd with $t \geq 1$. Begin with k_0 vertex-disjoint copies of K_2 in S , and include all their edges in M . Designate one copy as the root component R . A remaining degree slot of a vertex means an unassigned incident edge needed to bring its degree to three.

All attachments below follow one canonical pairing, so the construction is a single well-defined graph rather than a family of bijections. For a matched path $P_{2s} = v_1 \dots v_{2s}$, list remaining slots in vertex order along the path, repeating each endpoint once for every unused incidence: v_1, v_1 , then each internal vertex once, then v_{2s}, v_{2s} . In particular a copy of K_2 contributes the four-slot list v_1, v_1, v_2, v_2 . For a U -path $x_1 \dots x_r$ or U -cycle $z_1 \dots z_r$, list vertices in that path or cyclic order from the designated start. Pair the two lists in this order. New K_2 components are introduced in the order they are first used.

1. Place a path $P_4 = x_1x_2x_3x_4$ in U and attach x_1, x_2, x_3, x_4 to the four canonical slots of R in that order. Join x_1 and x_4 to two distinct unused K_2 components R_1 and R_4 , using one slot of each.
2. If $t = 1$, declare both R_1 and R_4 terminal. If $t > 1$, declare R_1 terminal and R_4 active. At each subsequent step, take the canonical slot list of the active K_2 , use its first slot for the already present upward edge to the previous U -path, and pair the remaining three slots with a new path $P_3 = y_1y_2y_3$. Join y_1 to a new unused K_2 (declared terminal) and y_3 to a new unused K_2 (declared the next active component). Repeat $t - 1$ times, then declare the final active component terminal.

3. Process terminal K_2 components in the order they were designated. Each has three remaining canonical slots; attach a triangle $z_1 z_2 z_3$ by pairing those slots with z_1, z_2, z_3 in order.

The procedure uses $1 + 2 + 2(t - 1) = 2t + 1 = k_0$ copies of K_2 and leaves $t + 1$ of them terminal. Every degree slot is used once, no loop or parallel edge is created, every vertex has degree three, and the attachment sequence keeps the graph connected.

The graph $G[S]$ is the disjoint union of k_0 copies of K_2 , so M is acyclic. The endpoints of every edge of the root P_4 meet R ; all vertices of each P_3 meet its active K_2 ; and all vertices of each triangle meet one terminal K_2 . Lemma 4.2 therefore blocks every edge of $G[U]$, so M is maximal. Moreover,

$$|U| = 4 + 3(t - 1) + 3(t + 1) = 6t + 4 = 3k_0 + 1, \quad n = 2k_0 + |U| = 5k_0 + 1.$$

6.2 Residue-class adjustment

The replaced leaf is the first designated terminal, namely the K_2 joined to x_1 . In the base construction that K_2 is joined to the rest of the graph by a single edge, and the other end of that edge is a vertex $w \in U$.

For an integer $q \geq 0$, replace this terminal K_2 by the alternating path $P_{2(q+1)} = v_1 \dots v_{2q+2}$ in S and replace the triangle by C_{2q+3} in U . Include the $q + 1$ alternating path edges in M . Form the canonical slot list of the path, which has $2q + 4$ entries. Reserve its first entry for the reattachment edge $v_1 w$, and pair the remaining $2q + 3$ slots with the cycle vertices $z_1, \dots, z_{2q+3}$ in cyclic order.

Because $w \in U$, the reattachment edge is a cut edge between S and U and not an edge inside S ; this is what keeps $G[S]$ a forest. Indeed the replacement piece contributes the single new F -component $P_{2(q+1)}$, which is a tree, and it is joined to the rest of S by no edge at all. The degree of w is unchanged, since the edge it previously received from the terminal K_2 is replaced by $v_1 w$. The operation therefore preserves simplicity, connectivity, cubicity, and the fact that $G[S]$ is a forest. Both endpoints of each new cycle edge meet the same S -path component, so Lemma 4.2 blocks every new H -edge and maximality is preserved; the blocking of every old H -edge is untouched, because each such edge is blocked through the root K_2 or an active K_2 , none of which is modified. Taking $q = 0$ recovers the original leaf piece, so the operation is consistent with the base construction. Figure 3 shows the resulting component-incidence skeleton for $t = 2$ and $q = 1$. Relative to $K_2 + C_3$, the replacement adds q matching edges and $4q$ vertices. If $k = k_0 + q$, then

$$n = (5k_0 + 1) + 4q = 5k + 1 - q. \tag{4}$$

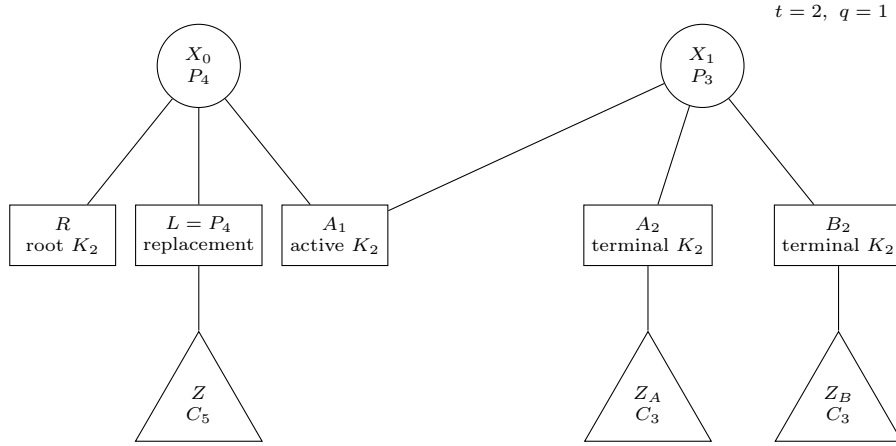


Fig. 3: Component-incidence skeleton for $t = 2, q = 1$. Circles are tree components of $H = G[U]$, rectangles are components of $F = G[S]$, and triangles are cycle components of H .

6.3 Covering every even order

Fix an even integer $n \geq 4$, set

$$k = \left\lceil \frac{n-1}{5} \right\rceil, \quad q = 5k + 1 - n.$$

The definition of the ceiling gives $k-1 < (n-1)/5 \leq k$, hence $5k-4 < n \leq 5k+1$ and $q \in \{0, 1, 2, 3, 4\}$. Put $k_0 = k - q$. Since $k_0 = n - 4k - 1$ and both n and $4k$ are even, k_0 is odd.

Only three cases occur. As $q \leq 4$ we have $k_0 \geq k - 4$, and $k \geq 1$ for every even $n \geq 4$. If $k_0 \leq -5$ then $q = k - k_0 \geq 6$, which is impossible; if $k_0 = -3$ then $q = k + 3 \leq 4$ forces $k = 1$ and hence $n = 5k + 1 - q = 2$, excluded by $n \geq 4$. Therefore $k_0 \in \{-1, 1\}$ or $k_0 \geq 3$, and these three cases are treated in turn.

If $k_0 \geq 3$, use the base construction and the size- q leaf replacement. Equation (4) gives exactly n vertices and k matching edges.

If $k_0 = 1$, then $q = k - 1 \leq 4$, so $k \leq 5$ and $n = 5k + 1 - (k - 1) = 4k + 2$, that is, $n \in \{6, 10, 14, 18, 22\}$. Take $F = P_{2k}$ with its alternating matching and $H = C_{2k+2}$, and pair the $2k + 2$ canonical slots of F with the cycle vertices in cyclic order. This graph has $4k + 2 = n$ vertices, and every H -edge is jointly blocked by the unique F -component. This family is not the $k_0 = 1$ instance of Equation (4): with no skeleton to reattach, the path keeps all $2k + 2$ of its slots instead of reserving one, so the attached cycle is C_{2k+2} rather than C_{2k+1} . The distinction is forced, since a piece $P_{2k} + C_{2k+1}$ would have the odd order $4k + 1$ and no cubic graph has odd order.

It remains to treat $k_0 = -1$, where $q = k + 1 \leq 4$ gives $k \leq 3$ and $n = 5k + 1 - (k + 1) = 4k$, that is, $n \in \{4, 8, 12\}$. For K_4 , any edge is a maximal

acyclic matching. For the cube Q_3 , label vertices by three-bit strings and take

$$M = \{000-001, 110-111\}.$$

For the hexagonal prism, let the two 6-cycles be $a_0 \dots a_5$ and $b_0 \dots b_5$ with vertical edges $a_i b_i$, and take

$$M = \{a_0 b_0, a_2 b_2, a_4 b_4\}.$$

In each case, direct application of Lemma 4.2 proves maximality.

Theorem 6.1. *For every even $n \geq 4$, there exists a connected simple cubic graph G of order n such that*

$$\mu'_{\text{ac}}(G) = \left\lceil \frac{n-1}{5} \right\rceil.$$

Consequently,

$$f_3(n) = \left\lceil \frac{n-1}{5} \right\rceil \quad (n \geq 4 \text{ even}).$$

Proof. The constructions above provide a maximal acyclic matching of size $k = \lceil (n-1)/5 \rceil$, so $\mu'_{\text{ac}}(G) \leq k$. Theorem 5.1 gives $\mu'_{\text{ac}}(G) \geq k$ for every connected cubic graph. $\square$

Remark 6.2. The constructions establish sharpness but do not classify the extremal graphs. At order 16 the census contains four graphs with $\mu'_{\text{ac}}(G) = 3$, and only three of them arise from the $t = 1$ base construction under some assignment of degree slots. The fourth replaces the root P_4 by a single unmatched vertex joined to three leaf pieces $K_2 + C_3$, one slot per piece. Both configurations realise $c = 3$ and $t = 1$ and hence the same value of (3); what differs is which tree component of H carries the incidences. Any classification of the extremal graphs must therefore account for at least these two mechanisms.

7 Algorithmic consequences: a theory-guided exact solver

This section translates the proved structure into Solver C, an exact algorithm specialized to nonempty connected simple cubic graphs. Solver C intentionally uses Theorem 5.1 and its counting consequences. It is therefore a *post-theorem* algorithm and must not replace Solvers A and B or the endpoint-set audit in the non-circular verification chain.

7.1 The edge-interaction forest

Let $e = ab$ and $f = cd$ be two candidate graph edges.

Definition 7.1. The pair (e, f) has:

- (i) an *endpoint conflict* if $\{a, b\} \cap \{c, d\} \neq \emptyset$;
- (ii) a *cycle conflict* if the endpoint sets are disjoint and

$$\lambda(e, f) := |E_G(\{a, b\}, \{c, d\})| \geq 2;$$

- (iii) a *link* if the endpoint sets are disjoint and $\lambda(e, f) = 1$.

For a set M of pairwise endpoint-disjoint edges, the *link graph* $L(M)$ has vertex set M and an edge ef for each linked pair.

Proposition 7.2 (interaction-forest characterization). *A set $M \subseteq E(G)$ is an acyclic matching if and only if:*

- (i) *no selected pair has an endpoint conflict;*
- (ii) *no selected pair has a cycle conflict; and*
- (iii) *$L(M)$ is a forest.*

Proof. The endpoint-conflict condition is precisely the requirement that M be a matching, so assume M is a matching and contract every edge of M in the induced graph $G[V(M)]$. Every vertex of $G[V(M)]$ is covered by M , so the contraction sends each pair of endpoints to a single vertex, and every remaining edge of $G[V(M)]$ joins two distinct such vertices.

Suppose first that M has a cycle conflict, say $\lambda(e, f) \geq 2$ for $e = ab$ and $f = cd$. Two distinct edges of $E_G(\{a, b\}, \{c, d\})$ together with e and f contain a cycle of $G[V(M)]$ of length three or four, so $G[V(M)]$ is not a forest. Suppose next that M has no cycle conflict but $L(M)$ contains a cycle $f_1 f_2 \dots f_s f_1$. Choosing the unique cross edge realising each link and traversing the matching edges f_i in between yields a cycle of $G[V(M)]$, so again $G[V(M)]$ is not a forest.

Conversely, suppose $G[V(M)]$ contains a cycle Q . Contracting maps Q to a closed walk W in $L(M)$ of positive length, since Q is not contained in a single matched pair. If W immediately backtracked, Q would use two distinct edges of $E_G(\{a, b\}, \{c, d\})$ for one pair, that is, a cycle conflict. In the absence of a cycle conflict, W is a closed walk without immediate backtracking and therefore contains a cycle of $L(M)$. Hence $G[V(M)]$ is a forest exactly when there is no cycle conflict and $L(M)$ is a forest. $\square$

The characterization also gives an exact maximality test. Suppose M is acyclic and $e \notin M$ has no endpoint or cycle conflict with a selected edge. Adding e inserts one new vertex into $L(M)$, adjacent to all selected edges linked to e . This preserves the forest exactly when those neighbors lie in pairwise distinct components of $L(M)$. Therefore M is maximal if and only if every unselected edge either has a hard conflict (endpoint or cycle), or has two link-neighbors already lying in the same link-forest component.

Algorithm 3 Solver C: theorem-guided interaction-forest search

Input: A nonempty connected simple cubic graph G

```

1: Precompute endpoint conflicts, cycle conflicts, and links
2:  $\ell \leftarrow \lceil (|V(G)| - 1)/5 \rceil$ 
3: for  $k = \ell, \ell + 1, \dots, \lfloor |V(G)|/2 \rfloor$  do
4:    $D \leftarrow \min\{k - 1, 5k + 1 - |V(G)|\}$ 
5:   if  $D < 0$  then
6:     continue
7:   else if  $D = 0$  then
8:      $M \leftarrow \text{ZERODEFECTDOMINATION}(G, k)$ 
9:   else
10:     $M \leftarrow \text{INTERACTIONFORESTSEARCH}(G, k, D)$ 
11:   if  $M \neq \text{failure}$  then
12:     Verify  $M$  with the definition-level matching/forest/maximality routine
13:   return  $(k, M)$ 

```

7.2 Rollback-DSU search

Solver C precomputes all endpoint conflicts, cycle conflicts, and links in $\mathcal{O}(m^2)$ time. During a branch, each selected graph edge is an active vertex of the link forest. A rollback disjoint-set union structure maintains its components. To add a candidate e , the algorithm checks whether two current link-neighbors of e already have the same root. If so, adding e closes a cycle; otherwise e is activated and joined to those distinct roots. A snapshot permits constant-stack rollback after the recursive call.

Search is performed in increasing target cardinality. Capacity bounds use the number of available suffix edges and their endpoint union. Near a leaf of the search tree, Solver C also applies a rigid-maximality rule. If an unselected edge can have at most one link-neighbor in every completion of the current branch, it can never be cycle-blocked. It must therefore be selected itself or hard-conflicted by a future selected edge. These necessary alternatives form small hitting-set constraints; the implementation invokes the rule only when at most two matching slots remain and solves the resulting hitting-set check exactly.

7.3 The theorem-derived defect budget

For an acyclic k -edge matching, let $\delta = |E(L(M))|$. Since $L(M)$ is a forest with k vertices, its component count is

$$c = k - \delta.$$

The variables c and t are the same component counts used in (3). From

$$n = 4k + 2(c - t) \quad \text{and} \quad c \leq 2t + 1,$$

we obtain $n \leq 4k + c + 1$, hence $c \geq n - 4k - 1$. Therefore

$$\delta = k - c \leq 5k + 1 - n.$$

Also $c \geq 1$, so $\delta \leq k - 1$. Every solution at target k must satisfy

$$\delta \leq \min\{k - 1, 5k + 1 - n\}. \quad (5)$$

At the first theoretically possible target $k_0 = \lceil (n - 1)/5 \rceil$, the quantity $5k_0 + 1 - n$ belongs to $\{0, 1, 2, 3, 4\}$. Thus the first target layer permits at most four link edges. This bound is safe but theorem-dependent; it is the principal reason Solver C cannot independently verify Theorem 5.1.

7.4 The zero-defect residue class

When $n = 5k + 1$, (5) forces $\delta = 0$. The selected edges then form an independent set in the union of the endpoint-conflict, cycle-conflict, and link relations: no two of them are related at all. Every component of $L(M)$ is a single vertex, so an unselected edge can never have two link-neighbors in one component, and a single link to a selected edge does not create a cycle. Maximality is therefore achieved only by selecting an edge itself or by selecting one of its endpoint or cycle hard conflicts.

Consequently the zero-defect target is an independent domination problem with one relation for incompatibility and a smaller relation for domination. Solver C branches on an undominated edge e and selects one member of

$$\{e\} \cup N_{\text{hard}}(e).$$

Every feasible solution must hit this set. Fail-first selection, a safe coverage lower bound, and memoization of failed (*available*, *dominated*, *slots*) states complete the specialized search. No polynomial-time claim is made: the general worst-case search remains exponential.

Proposition 7.3. *Solver C returns $\mu'_{\text{ac}}(G)$ and a corresponding maximal acyclic matching for every nonempty connected simple cubic graph G .*

Proof. Proposition 7.2 shows that the precomputed relations and the rollback structure recognize acyclic matchings exactly. Capacity pruning is necessary for completing the requested number of disjoint edges. The defect pruning follows from (5). Every rigid-maximality constraint is necessary because an edge with at most one possible link-neighbor cannot become cycle-blocked; the exact small hitting-set test therefore removes no completion. In the zero-defect case, branching on $\{e\} \cup N_{\text{hard}}(e)$ is complete for every undominated edge. The coverage bound is a standard lower bound on the remaining number of dominating choices, and memoization only reuses failed exact states.

At a complete candidate, the interaction criterion tests maximality exactly, and the implementation rechecks the returned witness with the independent definition-level routine. The search starts at the proved lower bound and increases k only after exhausting the previous target. Hence the first returned witness is optimal. $\square$

Table 5: Paired full-census comparison. Times are medians of whole-layer repetitions. Speedup is T_B/T_C , and reduction compares complete candidates.

n	Graphs	B time (s)	C time (s)	Speedup	B candidates	C candidates	Reduction	Disc.
4	1	0.0008	0.0010	$0.88\times$	2	1	50.00%	0
6	2	0.0053	0.0040	$1.33\times$	5	2	60.00%	0
8	5	0.0127	0.0126	$1.01\times$	70	5	92.86%	0
10	19	0.0839	0.0896	$0.94\times$	989	358	63.80%	0
12	85	0.4467	0.3775	$1.18\times$	11,151	179	98.39%	0
14	509	5.9727	4.8901	$1.22\times$	224,027	43,691	80.50%	0
16	4,060	94.1762	32.4600	$2.90\times$	4,832,310	40,158	99.17%	0
All	4,681	100.6984	37.8348	$2.66\times$	5,068,554	84,394	98.34%	0

7.5 A linear vertex kernel

Consider the parameterized decision problem on connected simple cubic graphs: given (G, K) , decide whether $\mu'_{\text{ac}}(G) \leq K$.

Corollary 7.4. *LOW-ACY-MATCHING on connected simple cubic graphs is fixed-parameter tractable with respect to the solution-size parameter K .*

Proof. If $\mu'_{\text{ac}}(G) \leq K$, then Theorem 5.1 implies $n \leq 5K + 1$. Thus any instance with $n > 5K + 1$ is a no-instance and may be replaced by a fixed no-instance, for example $(K_4, 0)$. Every remaining instance already has at most $5K + 1$ vertices. This is an identity kernel on yes-instances: the bound itself discards oversized inputs, and no further reduction rule is applied. A cubic graph on at most $5K + 1$ vertices has at most $3(5K + 1)/2$ edges, so enumerating edge subsets of size at most K and checking the definition takes $2^{\mathcal{O}(K)} \text{poly}(|G|)$ time. $\square$

The same bound is what Solver C uses as a search floor. It is recorded here as an FPT consequence of Theorem 5.1, not as an independent kernelization algorithm.

7.6 Census performance at a glance

Solver C is a post-theorem search procedure, not independent evidence for Theorem 5.1. On the complete census of 4681 non-isomorphic connected simple cubic graphs of orders 4, 6, $\dots$, 16 it agrees with the theorem-independent bitset solver on every optimum. Table 5 records whole-layer median times: at order 16 the median drops from 94.18 to 32.46 seconds and the number of complete candidates from 4,832,310 to 40,158. The timing protocol, per-graph ratios, equality-family scaling, and ablations are given in Section A.

Table 6: Summary of independent implementations and machine checks.

Verification item	Scale or result
Independent graph generators	GENREG and nauty 2.9.3 geng
Canonical graph-set comparison	1, 2, 5, 19, 85, 509, 4060 classes at orders 4 through 16; identical multisets at every order
Solver A versus Solver B	Graph-by-graph agreement through $n = 12$
Independent optimum audit, $n = 14$	All 509 graphs: 418 have value 3 and 91 have value 4
Independent optimum audit, $n = 16$	All 4060 graphs: 4 have value 3 and 4056 have value 4
Lower-value exclusion	Every smaller endpoint-set cardinality was exhausted
Solver C interaction audit	Definition-level interaction characterization checked on K_4 , $K_{3,3}$, the cube, Petersen graph, and a 10-vertex prism
Solver B versus Solver C	All 4681 census graphs plus 64 gate instances; zero discrepancies
Current package test suite	12 tests passed

8 Independent computational verification and reproducibility

8.1 Independent implementations and structural stress tests

Solvers A and B agree graph by graph on every cubic graph of order at most 12. The test suite also covers K_4 , $K_{3,3}$, paths, cycles, non-cubic graphs, small graph-atlas instances, and a dedicated trap instance that requires rebuilding the full vertex-induced subgraph after a matching edge is added.

The order-14 and order-16 audit uses a different search space. For each even-cardinality set $S \subseteq V(G)$, considered in increasing $|S|$, it checks by disjoint-set union whether $G[S]$ is a forest, recursively searches for a perfect matching of $G[S]$, and tests every edge uv outside S by rebuilding $G[S \cup \{u, v\}]$. A k -edge acyclic matching has an endpoint set of size $2k$ satisfying the first two tests; conversely, a perfect matching of such a set is acyclic. The last test is equivalent to maximality by Lemma 4.1. The first accepted endpoint set therefore certifies the optimum, not merely a witness value.

The endpoint-set audit has its own **graph6** decoder and does not call the matching enumeration, acyclicity, maximality, or cache routines of Solvers A or B. Its vertex-set formulation and recursive perfect-matching test avoid the principal control flow shared by the two edge-based solvers.

Solver C received separate definition-level interaction audits. Across the five standard cubic graphs in Table 6, all tested edge subsets agree on matching feasibility, acyclicity, and maximality. The publication benchmark adds 64 gate instances and graph-by-graph agreement on the complete 4681-graph census. These checks validate the implementation but, because Solver C uses the theorem-derived defect budget, they are not independent evidence for Theorem 5.1.

Table 7: Environment of the publication Solver B–C benchmark.

Item	Recorded value
Operating system	Microsoft Windows 11, build 26200, 64 bit
Processor	AMD Ryzen 7 7435H; 8 physical and 16 logical cores
Memory	15.74 GiB RAM
Python	CPython 3.11.7, Anaconda 64 bit, MSC v.1916
Graph library	NetworkX 3.1 Hagberg et al. (2008)
Graph generator	nauty 2.9.3 geng ; connected, minimum/maximum degree 3
Execution	Single-threaded; <code>time.perf_counter</code> ; no concurrent jobs
Repetitions	Five per complete layer for $n \leq 12$; three for $n = 14, 16$
Integrity	SHA-256 recorded for Solver B, Solver C, checker, and every census file
Benchmark date	19 August 2026

8.2 Environment and reproducibility materials

The publication performance run used the environment in Table 7. Both solvers were single-threaded, no other benchmark jobs ran concurrently, and the same interpreter, graph objects, timer, and input order were used. The nauty generator was compiled with GCC under Ubuntu/WSL, while the timed solver calls ran natively on Windows. The separate endpoint-set proof audit used its previously frozen environment and intentionally did not import NetworkX.

The reproducibility materials record the raw and canonical graph collections, order-specific comparison reports, complete order-14 and order-16 optimum tables, the independent **graph6** decoder and endpoint-set checker, definition-level routines, Solvers A–C, structural certificates, equality constructors, tests, commands, software versions, source and census SHA-256 values, raw repetition times, and all 4681 per-graph comparison rows. Source code is available at xxx.

The lower bound and equality construction do not depend on the census. A single explicit graph with a valid matching certificate suffices to refute the empirical quarter benchmark, while the exact extremal formula follows entirely from Sections 4 to 6. Computation discovers counterexamples, generates structural hypotheses, and supplies machine-checkable certificates.

8.3 Threats to validity

- **Graph-generation independence.** GENREG and **geng** use different generation algorithms and produce identical canonical multisets. They share `labelg` only as a post-generation canonicalizer; neither generator uses it to decide which graphs to emit.
- **Optimum verification.** At orders 14 and 16, the endpoint-set checker exhausts every smaller cardinality and uses separate parsing, forest, perfect-matching, and extension code. Reproducibility still depends on preserving source, commands, inputs, and hashes.

- **Proof dependence of Solver C.** Solver C uses (5), which is derived from the main theorem. Its agreement with Solver B is an implementation audit and an algorithmic consequence, not a non-circular verification of the theorem.
- **Performance scope.** The full-census comparison is a paired single-machine benchmark, not a portable performance ranking. Only one canonical graph ordering, one Python and NetworkX stack, and one hardware platform were tested. Timings below a millisecond are particularly sensitive to interpreter and timer noise.
- **Equality-family censoring.** Equality orders above 28 use one run, and Solver B timings at orders 46 and 56 are right-censored at 300 seconds. Their large structured-instance gains are reported separately and are not substituted for census averages.
- **Solver scale.** The complete census stops at order 16, and no SAT or ILP certificate generator has been applied to the full collection at larger orders. The proof is independent of that scale limitation.
- **Literature scope.** Results for the maximum acyclic matching number do not settle the minimum-maximal problem studied here. In particular the bound $\nu_{\text{ac}}(G) \geq n/4$ of Fürst and Rautenbach (2018) bounds a parameter that dominates $\mu'_{\text{ac}}(G)$, so it neither implies nor contradicts the present theorem; the census in Section 3 shows that the two parameters do in fact separate on cubic graphs.

9 Discussion and open questions

The extremal formula resolves the fixed-order minimum over connected cubic graphs, and Corollary 7.4 records the resulting fixed-parameter tractability for the solution-size parameter in this class. It does not settle the classical per-instance complexity or the behavior under stronger restrictions.

- **Cubic complexity.** What is the classical complexity of deciding $\mu'_{\text{ac}}(G) \leq K$ on connected cubic graphs? The known hardness for bipartite graphs of maximum degree six and 4-regular graphs does not resolve this class.
- **Exact exponential algorithms.** Can the interaction-forest formulation yield a rigorously improved worst-case exponential base, beyond the practical pruning used by Solver C?
- **Kernel size.** The reduction in Corollary 7.4 is an identity kernel: yes-instances are left unchanged. Can a strictly smaller kernel be obtained by genuine reduction rules, and are there conditional or unconditional lower bounds on the best possible kernel size?
- **Bounded-width algorithms.** Maximal acyclic matching feasibility and cardinality minimization are expressible in LinEMSO_2 , so the meta-theorem of Arnborg et al. (1991) gives fixed-parameter tractability for treewidth. Can one obtain an explicit single-exponential dynamic program with matching ETH or SETH lower bounds?

- **Verifiable computation.** Can compact SAT, ILP, or CP-SAT formulations produce independently checkable infeasibility certificates and outperform generic edge-subset enumeration?
- **Restricted cubic classes.** What are the best lower bounds for 3-edge-connected, bipartite, or large-girth cubic graphs?
- **Extremal classification.** Can all graphs attaining $\lceil (n-1)/5 \rceil$ be characterized, rather than one explicit equality family? Remark 6.2 shows that at order 16 the extremal graphs already split into two structural types, distinguished by whether the unique tree component of H is the root P_4 or an isolated vertex. A classification would have to enumerate the admissible component-incidence trees realising $c = 2t + 1$.
- **Higher regular degree.** For fixed $r \geq 4$, what are the asymptotic constant and exact remainder term of the analogous extremal function over connected r -regular graphs?

10 Conclusion

We determined the exact extremal behavior of minimum maximal acyclic matchings in connected cubic graphs. Analysis of the unmatched induced subgraph and its incidence with the forest induced by matching endpoints yields the sharp parity-sensitive bound, and explicit constructions attain it at every even order. The complete order-16 census exposes exactly four counterexamples to the quarter benchmark inherited from the maximum acyclic matching number; three of them are realised by the equality family and the fourth is not, so sharpness and classification are genuinely different questions. Independent graph generation and three theorem-independent exact search routes separate computational discovery from mathematical proof.

The structural theorem also has direct algorithmic consequences. Contracting selected matching edges yields an exact edge-interaction forest, the counting proof supplies a defect budget of at most four at the first feasible target, and deciding $\mu'_{\text{ac}}(G) \leq K$ on connected cubic graphs is fixed-parameter tractable because every yes-instance has at most $5K + 1$ vertices. These consequences strengthen the computational-combinatorics aspect of the result without changing the central conclusion:

$$f_3(n) = \left\lceil \frac{n-1}{5} \right\rceil \quad \text{for every even } n \geq 4.$$

A Solver C benchmark details

We compared the theorem-independent bitset solver (Solver B) with Solver C on the complete census of 4681 non-isomorphic connected simple cubic graphs of orders $4, 6, \dots, 16$. The census was generated with nauty 2.9.3 and its layer sizes were checked as 1, 2, 5, 19, 85, 509, and 4060. Before timing, the implementations were required to agree on 64 gate instances: ten standard or prism graphs, the four order-16 counterexamples, and 50 fixed-seed random

Table 8: Equality-family comparison under a common 300-second timeout. The column D is the defect budget $\min\{k-1, 5k+1-n\}$ of (5); rows with $D=0$ use the specialised zero-defect routine. Ratios above 100 are reported only as magnitudes. A dash denotes a speedup censored by a Solver B timeout.

n	$\mu'_{ac}(G)$	D	B time (s)	C time (s)	Speedup	Status
16	3	0	0.0143	0.00525	$2.7\times$	both solved
20	4	1	0.0965	0.02424	$4.0\times$	both solved
24	5	2	0.8888	0.15152	$5.9\times$	both solved
26	5	0	1.5255	0.01000	$>10^2\times$	both solved
28	6	3	8.4682	0.96229	$8.8\times$	both solved
32	7	4	89.2176	6.44114	$13.9\times$	both solved
36	7	0	290.5094	0.01720	$>10^4\times$	both solved
46	9	0	timeout	0.02144	–	B timeout
56	11	0	timeout	0.02971	–	B timeout

connected cubic graphs. Every returned witness was also checked by the definition-level maximal-acyclic-matching routine. The gate and the complete census had zero discrepancies.

Graphs were decoded before each timed stage. Each solver received the same in-memory graphs in the same canonical **graph6** order. After an untimed warm-up, every complete layer through order 12 was repeated five times and the order-14 and order-16 layers were repeated three times. Solver order was alternated between repetitions, garbage collection was disabled only within individual solver calls, and `time.perf_counter` measured solver time. Table 5 reports medians of whole-layer totals. The search columns use the number of complete candidates examined by each implementation; Solver C recursion nodes are retained separately in the archived per-graph data and are not substituted for this common leaf-level measure.

The whole-census aggregate in Table 5 is a sum of the seven layer medians. Per-graph ratios answer a different question: across all 4681 graphs, their median and geometric mean are $4.41\times$ and $3.81\times$. At order 16 they are $4.58\times$ and $4.46\times$, with an interquartile range of $4.00\text{--}5.11\times$ and a range of $1.08\text{--}8.22\times$. Solver C is faster on every order-16 graph. It is slower on 127 smaller graphs, all of order at most 14, where interpreter, preprocessing, and pruning overhead are large relative to the underlying search.

Grouping within an order is consistent with the intended role of the defect budget. At order 10 the geometric-mean speedups for defect budgets one and at least five are $1.09\times$ and $0.70\times$; at order 14 the corresponding figures for budgets two and at least five are $1.44\times$ and $1.14\times$. Only four order-16 census graphs have zero defect, so their $5.23\times$ geometric-mean speedup is suggestive rather than a basis for a broad claim.

The complete census is the primary performance experiment. Separately, we tested the equality construction at orders 16, 20, 24, 26, 28, 32, 36, 46, and 56 with a common 300-second timeout. Orders through 28 used three repetitions and report medians; larger orders used one run. Table 8 records these structured scalability results.

The five rows with $D=0$ are exactly the orders with $n=5k+1$, namely $n\equiv 6\pmod{10}$, where the budget of (5) collapses and Solver C enters the specialised domination routine. On those rows Solver C takes between 0.005 and 0.030 seconds across the whole range $16\leq n\leq 56$, whereas Solver B grows from 0.014 seconds to a timeout. The extreme ratios therefore track the blow-up of the unguided baseline rather than any variation in Solver C, which is also why the same zero-defect mode yields $2.7\times$ at order 16 and more than $10^4\times$ at order 36. These are structured members of the equality family, not a random sample, and the ratios must not be read as average census speedups.

Finally, five Solver C variants were run three times on a deterministic, stratified sample of 100 order-16 graphs. On the four zero-defect graphs, replacing the specialized zero-defect routine by the generic search increased median search time by approximately $9.7\times$, while disabling the theorem-derived budget increased it by approximately $15.2\times$. Failed-state

memoization had no measurable benefit on these four found-solution searches. On the remaining 96 graphs, removing near-leaf maximality pruning increased the total recursion-node count from 1853 to 3268, although the additional pruning test did not improve median microbenchmark time. These ablations identify mechanisms, not portable worst-case guarantees.

References

- Arnborg S, Lagergren J, Seese D (1991) Easy problems for tree-decomposable graphs. *Journal of Algorithms* 12:308–340. [https://doi.org/10.1016/0196-6774\(91\)90006-K](https://doi.org/10.1016/0196-6774(91)90006-K)
- Cameron K (1989) Induced matchings. *Discrete Applied Mathematics* 24:97–102. [https://doi.org/10.1016/0166-218X\(92\)90275-F](https://doi.org/10.1016/0166-218X(92)90275-F)
- Chaudhary J, Panda BS (2021) On the complexity of minimum maximal uniquely restricted matching. *Theoretical Computer Science* 882:15–28. <https://doi.org/10.1016/j.tcs.2021.05.036>
- Chaudhary J, Zehavi M (2025) p -matchings parameterized by treewidth. *SIAM Journal on Discrete Mathematics* 39:1280–1311. <https://doi.org/10.1137/23M160013X>
- Chaudhary J, Mishra S, Panda BS (2024) On the complexity of minimum maximal acyclic matchings. *Journal of Combinatorial Optimization* 48:10. <https://doi.org/10.1007/s10878-024-01200-3>
- Chaudhary J, Mishra S, Panda BS (2025) Minimum maximal acyclic matching in proper interval graphs. *Discrete Applied Mathematics* 360:414–427. <https://doi.org/10.1016/j.dam.2024.10.012>
- Edmonds J (1965) Paths, trees, and flowers. *Canadian Journal of Mathematics* 17:449–467. <https://doi.org/10.4153/CJM-1965-045-4>
- Fürst M, Rautenbach D (2018) A lower bound on the acyclic matching number of subcubic graphs. *Discrete Mathematics* 341(8):2353–2358. <https://doi.org/10.1016/j.disc.2018.05.010>
- Fürst M, Rautenbach D (2019) On some hard and some tractable cases of the maximum acyclic matching problem. *Annals of Operations Research* 279:291–300. <https://doi.org/10.1007/s10479-019-03311-1>
- Goddard W, Hedetniemi SM, Hedetniemi ST, Laskar R (2005) Generalized subgraph-restricted matchings in graphs. *Discrete Mathematics* 293:129–138. <https://doi.org/10.1016/j.disc.2004.08.027>
- Golumbic MC, Hirst T, Lewenstein M (2001) Uniquely restricted matchings. *Algorithmica* 31:139–154. <https://doi.org/10.1007/s00453-001-0004-z>
- Hagberg AA, Schult DA, Swart PJ (2008) Exploring network structure, dynamics, and function using NetworkX. In: *Proceedings of the 7th Python in Science Conference*, pp 11–15
- Lovász L, Plummer MD (2009) *Matching Theory*. American Mathematical Society, Providence
- McKay BD, Piperno A (2014) Practical graph isomorphism, II. *Journal of Symbolic Computation* 60:94–112. <https://doi.org/10.1016/j.jsc.2013.09.003>
- Meringer M (1999) Fast generation of regular graphs and construction of cages. *Journal of Graph Theory* 30:137–146. [https://doi.org/10.1002/\(SICI\)1097-0118\(199902\)30:2<137::AID-JGT7>3.0.CO;2-G](https://doi.org/10.1002/(SICI)1097-0118(199902)30:2<137::AID-JGT7>3.0.CO;2-G)
- Panda BS, Chaudhary J (2023) Acyclic matching in some subclasses of graphs. *Theoretical Computer Science* 943:36–49. <https://doi.org/10.1016/j.tcs.2022.12.008>
- Yannakakis M, Gavril F (1980) Edge dominating sets in graphs. *SIAM Journal on Applied Mathematics* 38:364–372. <https://doi.org/10.1137/0138030>

Statements and Declarations

Funding. The author declares that no funds, grants, or other support were received during the preparation of this manuscript.

Competing interests. The author has no relevant financial or non-financial interests to disclose.

Author contributions. Hao Tang conceived the study, implemented the algorithms, designed and conducted the experiments, performed the structural analysis, developed the proofs, and wrote the manuscript.

Data and code availability. Source code is available at xxx.

Use of AI-assisted tools. AI-assisted tools were used for exploratory prompts, code review, and language polishing. They were not listed as authors and did not assume responsibility for mathematical or computational claims. The author takes full responsibility for the final manuscript.